But just limiting to swaps doesn't tell us the whole story: in section 2.9 we delve into the performance analysis of these two methods, and take into consideration also the number of array accesses, or equivalently the number of comparisons on heap elements, that are needed for each method. While `insert` accesses only one element per heap's level, method `top` traverses the tree from root to leaves, and at each level it needs to go through the list of children of a node: therefore, it needs approximately up to `D*logD(n)` accesses, on a heap with branching factor `D` and containing `n` elements.

Table 2.6 Main operations provided by a d-ary heap, and the number of swaps (assuming `n` elements)

Operation	Number of swaps	Extra Space
Insert	$\sim \log_D(n)$	O(1)
Top	$\sim \log_D(n)$	O(1)
Heapify	~n	O(1)

Tables 2.6 and 2.7 summarize the performance analysis for the three main methods in heaps' API.

So, for method `top`, a larger branching factor doesn't always improve performance, because while `logD(n)` becomes smaller, `d` becomes larger. Bringing it to an extreme, if we choose `D > n-1`, then the heap becomes a root with a list of `n-1` children, and so while `insert` will require just `1` comparison and `1` swap, `top` method will need `n` comparisons